Exploring Pro-Code Extension of a Zoho Creator Application using Zoho Catalyst / Custom Functions

Roshan S

241501170

AIML

Abstract

This experiment explores how a low-code Zoho Creator application is extended with code. Custom Deluge functions were created in the Workflow area of the GYM MEMBERSHIP application, the developer tools of the platform (Application IDE, Widgets, Variables) were reviewed, and an extension using Zoho Catalyst is designed in which a serverless function calculates the membership fee for an enrollment and is called from Creator.

1. Aim

To explore pro-code extension of a Zoho Creator application using Zoho Catalyst and custom functions.

Note on Execution

Zoho Catalyst and the connection to it need a paid plan. If the lab account is on a free or expired trial, the experiment is documented as a procedure together with the evidence that could be captured, and steps that could not be executed are marked as such.

2. Concepts

Custom functions. Reusable Deluge functions with a name, language, return type and syntax, kept in the Functions tab of the Workflow area and called from workflows, buttons and schedules.

Developer tools. The Settings page lists Developer Tools: Schema Builder, Application IDE, Widgets and Variables. Widgets let developers embed custom HTML, CSS and JavaScript, and the Application IDE gives a code view of the whole application.

Zoho Catalyst. A serverless platform on which functions are written in languages such as Node.js, Java or Python and exposed over HTTPS. A Creator application calls such a function with invokeUrl through a connection, which extends the application beyond what Deluge alone can do.

Use case. A Catalyst function calculates the total membership fee from the number of months in the plan, the monthly rate and the GST percentage, and Creator displays or stores the result with the enrollment.

3. Procedure

1. Open the Workflow area and the Functions tab, and create custom Deluge functions in the Default namespace (Figure 1: CountQuarterly() and QuarterlyMembers()).
2. Open Settings and review the Developer Tools: Schema Builder, Application IDE, Widgets and Variables (Figure 2).
3. Create a Catalyst project and an Advanced I/O function with the Node.js code in section 4, and deploy it (design).
4. In Creator, create a connection to the Catalyst function and call it with invokeUrl from a form workflow, then store or show the returned total (design).

4. Deluge / Configuration Code

```
// Zoho Catalyst Advanced I/O function (Node.js): membership fee calculator
const express = require("express");
const app = express();
app.get("/fee", (req, res) => {
  const months = parseInt(req.query.months);
  const rate = parseFloat(req.query.rate);
  const tax = parseFloat(req.query.tax);
  const total = months * rate * (1 + tax / 100);
  res.json({ months: months, total: total });
});
module.exports = app;

// Deluge in Zoho Creator: call the Catalyst function after an enrollment
resp = invokeUrl
[
    url : "https://<your-catalyst-function-url>/fee?
months=3&rate=1500&tax=18"
```

```
    type : GET
    connection : "catalystconn"
];
info resp.get("total");
```

5. Implementation Screenshots (GYM MEMBERSHIP application)

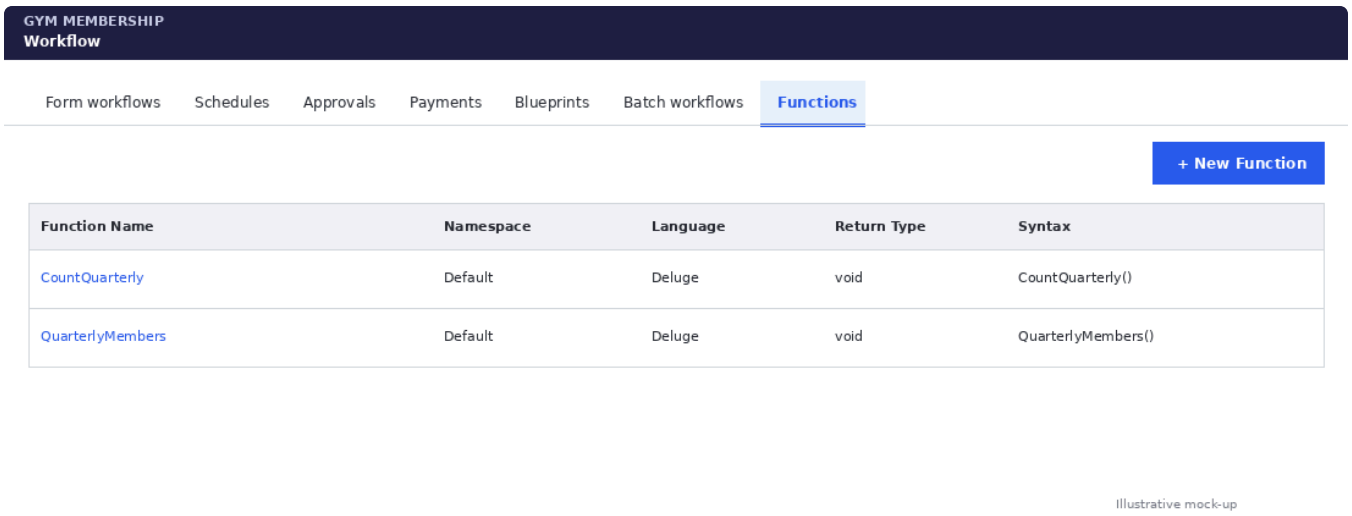


Figure 1: Functions tab of the Workflow area with the custom Deluge functions CountQuarterly() and QuarterlyMembers()

6. Expected Output

The Catalyst function returns the total for the given months, rate and tax as JSON, for example 5310 for 3 months at 1500 with 18 percent GST (3 × 1500 × 1.18), and Creator uses that value in the enrollment record or confirmation message.

7. Result

Custom Deluge functions were created in the application and the developer tools were reviewed. The Catalyst function and its call from Creator are given as a design with code; where Catalyst and the connection need a paid plan that the lab account does not have, they could not be executed.

8. Learning Outcome

Learnt how custom functions and the developer tools extend a Creator application with code, how a Zoho Catalyst serverless function can be called from Deluge through a connection, and when pro-code extension is worth using over pure low-code.